Reading the Architecture off the Weights: Recovering Model Anatomy from Tensor Manifests without Downloading Weights or Running Code

Sattaya Singkul
Researcher
joeysattaya@gmail.com

Abstract

Before anyone downloads, quantizes or deploys a model, they want to know what is inside it: which blocks repeat, what each block computes, how the components connect, and where the bytes go. Today that knowledge lives in model cards, per-family configuration schemas, or model code that must be executed. We show that most of it can be recovered from a far smaller and safer artifact: the tensor manifest, the list of tensor names, shapes and dtypes that every safetensors file stores in a header of a few tens of kilobytes. We present an algorithm that turns a manifest into an architectural description in four steps: it normalizes quantized storage back to logical tensors, finds repeated blocks in the module tree and types them by structural signature, labels the regions inside each block with a set of shape rules, and connects components by matching their residual widths. To measure it we build a benchmark of 511 architectures instantiated from the `transformers` library on the meta device, with ground truth taken from module classes that the algorithm never sees. Per-tensor role accuracy is 96.4% averaged over architectures, against 69.2% for name-pattern rules; with every name replaced by a random token it is still 94.7%. Logical parameter counts are exact for all 511 models, also under NVFP4, FP8 and GPTQ storage. We also state what a manifest cannot reveal (the split between head count and head size, causal masking, top-k routing) and keep such fields marked as guesses. On nine models released in 2026, including a 121B hybrid Mamba–MoE vision-language model, two speech transducers, an NVFP4 embedding model and a video diffusion transformer, the recovered anatomies agree with the published configurations. The method runs in the browser in about 13 ms per model and ships as an open tool.

1 Introduction

A deployment engineer handed a new checkpoint asks the same questions every time. How many layers, and are they all the same? Is it a mixture of experts, and how much of the size is experts? Which part runs once per input and which part runs once per generated token? Where will 4-bit quantization hurt? For a model family one already knows, `config.json` answers these. For anything new it does not: configuration keys differ across families (`hidden_size`, `d_model`, `n_embd`), composite models nest several schemas, and fields such as `layers_block_type` only make sense once you have read the modeling code. Reading that code, or running it with `trust_remote_code`, is slow and a known supply-chain risk on public model hubs [31]. Graph visualizers such as Netron [22] or Model Explorer [7] need an exported graph and, in practice, the full file.

There is a smaller artifact that almost every modern checkpoint already carries. A safetensors file [13] begins with an 8-byte length and a JSON header that lists every tensor's name, dtype, shape and offset. Over HTTP the header costs two range requests. Across the 511 architectures studied here it has a median size of 47 KB, which is 6×10^{-5} of the weights in BF16 (Figure 3). PyTorch zip checkpoints offer the same information in their `data.pkl` record, which can be read without touching the tensor data.

This paper asks how much of a model's architecture can be read from that manifest alone, and how reliably. The question has a practical side, since a tool that answers it can explain any hub model in a browser tab, and an analytical side: some properties are simply not present in shapes, and it is worth saying which.

Contributions.

- We formulate *manifest-only architecture recovery* and define an output that is useful to people: components and the data path between them, stacks of repeated blocks with their types, regions inside each block with their roles, and logical sizes under quantized storage (§3).
- We give an algorithm built on two kinds of evidence. Structure comes from the module tree: integer-indexed lists, their signatures, and the module that owns them. Function comes from *shape algebra*: the residual width shared by a block's norms, projections that leave and return to that width, expert tensors whose leading dimension agrees with a router, depthwise convolutions next to per-head state vectors, gate stacks of 3 or 4 times a hidden size, and modulation matrices of $k \cdot d$ outputs. Components are joined where one component's width (or its k^2 multiple after pixel shuffling) equals another's input (§4).
- We build a benchmark of 511 unique architectures with exact manifests and independent ground truth, and evaluate role labels, block discovery, block typing, hyperparameters and logical size, with name-obfuscation and quantization stress tests (§5, §6).
- We analyse what manifests cannot identify, and report case studies on nine models released in 2026 (§7). The implementation runs client-side in an open tool [25].

2 Related work

Model visualization. Netron [22], TensorBoard and Model Explorer [7] render operator graphs from exported formats; they show what is in the graph but do not group operators into semantic parts, and they need the file. Transformer Explainer [2] teaches one fixed architecture very well. Our goal sits between the two: semantic parts for arbitrary architectures, from a few kilobytes.

Architecture extraction. Side-channel work recovers architectures from cache timing or memory traces of a running model [12, 28]. There the adversary lacks the file; here the file's header is public and the challenge is interpretation, not observation. Weight-based fingerprinting such as HuRef [30] identifies a model's lineage from weight values; we never read values.

Documentation and supply chain. Model cards [16] describe intended use, but architectural fields are free text and often incomplete. Pickle-based formats can execute code on load [31]; safetensors avoids this by design, and our reader for PyTorch zip files parses only the pickle opcodes needed to rebuild names and shapes, never calling into Python objects.

Architectures covered. The rules in §4 encode the shape signatures of attention and its variants (grouped-query [1], latent attention [4]), sparse mixtures of experts [5, 24], selective state-space and linear-attention mixers [3, 9, 29] and their hybrids [17], conformers [10, 21], transducers [8, 27], diffusion transformers [19] and tabular in-context learners [11, 20]. For storage we handle GPTQ [6], AWQ [14], FP8 [15], MXFP4 [23] and NVFP4 [18] layouts.

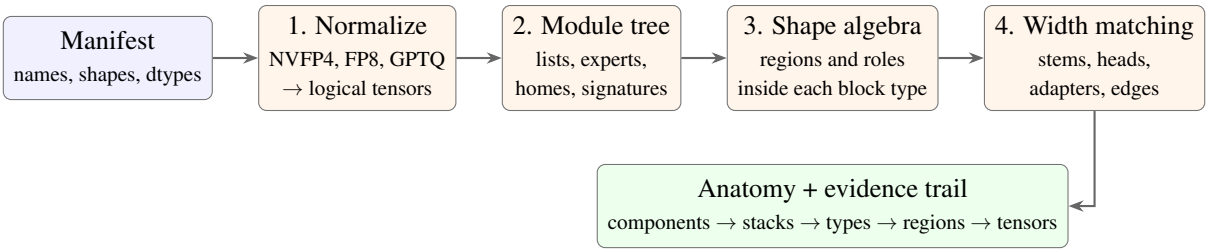


Figure 1: Overview. Each step records the evidence behind its decision; the tool shows it next to the result.

3 Problem

A manifest is a set $\mathscr{M} = \{(n_i, s_i, \tau_i)\}$ of tensor names (dot-separated paths), shapes and storage dtypes. We want an *anatomy* $\mathscr{A}$ with:

- **components** (e.g. vision tower, projector, language backbone, prediction network), each with a residual width d, a modality and a role, and directed **edges** for the data path;
- for each component, **stacks** of repeated blocks, each block assigned a **block type**;
- for each block type, **regions** with a role from a fixed taxonomy: attention, cross-attention, feed-forward, mixture of experts, SSM/linear attention, convolution module, adaLN modulation, recurrent cell, norm; and outside blocks, input stem, output head and adapter;
- per-tensor **roles** (query, router, expert, ...), **logical shapes** under quantized storage, and derived quantities: parameters, stored bytes, KV-cache bytes per token, active parameters per token.

What a manifest cannot say. Several properties are invisible in shapes, and an honest tool should keep them apart from what it measured. (i) *Head count versus head size.* Projections $W_q \in \mathbb{R}^{Hh \times d}$ with H heads of size h have the same shape as $H/2$ heads of size $2h$. The pair is identifiable only through per-head parameters (query/key norms of width h, relative-position biases of shape $H \times h$); otherwise only the product and the query/KV ratio are. (ii) *Masking.* A decoder-only language model with tied embeddings stores exactly the tensors of a bidirectional encoder of the same size; causality lives in code. (iii) *Routing and activation.* The number of experts is visible, the number used per token is not. Gated and ungated feed-forwards differ in matrix count, but SiLU and GELU do not. (iv) *Order inside a block.* Pre-norm and post-norm blocks have identical tensors. Our tool marks fields of type (i) as guesses and resolves (ii) and (iii) from `config.json` when it exists. The benchmark measures how often a reasonable prior is right.

4 Method

Figure 1 shows the pipeline. Every rule below uses shapes and tree structure; a small lexicon of name fragments (`attn`, `mlp`, `experts`, ...) is consulted only to break ties the shapes leave open, and can be switched off.

4.1 Normalizing storage

Quantized checkpoints store a weight as several tensors. We group tensors by module and recognize layouts by dtype and shape before looking at names. NVFP4 stores a `U8` tensor of shape $[o, i/2]$ with an FP8 scale tensor $[o, i/16]$ and a scalar second-level scale; the logical shape is $[o, i]$ and the group size follows from the ratio. FP8 stores an FP8 weight with a scalar, per-channel $[o, 1]$ or block scale. GPTQ stores `qweight`

as `I32` $[i/8, o]$ with scales $[i/g, o]$; AWQ packs the other axis, and the two are told apart by which axis the scales share. MXFP4 and MLX layouts are handled the same way. Auxiliary tensors are folded into their weight: they count toward stored bytes but not toward parameters.

4.2 Blocks from the module tree

Names form a tree. A node whose children are all integers (or keys like `expert_0`, `expert_1`, ...) is a *list*. An item is *composite* if its subtree holds at least two parameterized modules, which separates blocks from `nn.Sequential` wrappers around single layers.

Expert lists are lists nested inside the item of another list whose items are structurally identical two- or three-matrix MLPs without norms, usually next to a router matrix whose first dimension equals the number of items. Stacked experts appear instead as 3-D tensors $[E, \cdot, \cdot]$ in which one trailing dimension equals the block width.

Block lists are the innermost composite lists that are not expert lists, with one correction: when every item of an outer list holds exactly one short list of heterogeneous items (T5's `block.`i`.layer.`j), the inner items are sub-layers and the outer items are the blocks.

Each block list belongs to a *home*, the module that owns it, raised through ancestors as long as no other stack lives there. Homes become components; embeddings and heads next to an `encoder` wrapper therefore join the right component. Within a list, each block gets a signature: the sorted multiset of (relative path with indices masked, tensor rank). Identical signatures form a *block type*. In a hybrid stack such as Nemotron-H [17] this yields one type each for Mamba-2, MoE and attention layers, and the sequence of types is the layer pattern.

4.3 Regions by shape algebra

For each block type we read one representative block. Its residual width d is the length of the vectors held by its leaf norm modules, which is more reliable than counting matrix dimensions in blocks where the inner width of an SSM dominates. The block's children are then classified. Let $\mathscr{W}$ be the 2-D tensors of a child, and call a matrix an *input* if its second dimension is d and an *output* if its first is.

- **Mixture of experts:** stacked 3-D experts or an expert list, with a router $[E', d]$ where $E \le E' \le E + 8$ (some routers carry extra slots).
- **SSM / linear attention:** a depthwise convolution $[C, 1, k]$ with $k \le 8$ together with recurrence parameters that sit directly on the mixer module: at least two vectors of a common length $n_h \neq d$ (A, D, Δt bias), or a 2-D/3-D state tensor whose last dimension is a small state size. Mixers without a convolution (xLSTM, RWKV) are recognized by two or more per-head gate projections $[\le 64, d]$ or by decay vectors.
- **Feed-forward:** one output $[d, f]$ and one or two inputs $[f, d]$, or a fused input $[2f, d]$ with $f > 1.3d$.
- **Attention:** one output $[d, x]$ and one or more inputs, or one fused input $[x + 2x_{kv}, d]$. Inputs whose own input width is produced by another matrix in the region are internal (the low-rank chains of latent attention [4]); a region whose external inputs include d and at least two projections from a different width $w \ge 128$ is *cross-attention* from the component of width w. Vectors in norm-only submodules whose length divides x give the head size; parameters of shape $[H, h]$ with $Hh = x$ give both.
- **Recurrent cell:** two or more matrices whose rows are $3h$ or $4h$ (GRU, LSTM) with h among their columns, none returning to d.
- **Convolution module:** a depthwise convolution with pointwise $[C, C', 1]$ convolutions or batch-norm statistics.

Algorithm 1 Manifest-only anatomy recovery (sketch)

Require: manifest $\mathscr{M}$
$\mathscr{T} \leftarrow$ NORMALIZE($\mathscr{M}$) ▷ logical shapes, stored bytes
build the module tree; find lists, expert lists, block lists, homes
for all block list L **do**
  group items by structural signature $\rightarrow$ block types; pattern $\leftarrow$ type sequence
  for all block type β **do**
    $d \leftarrow$ width of leaf norms in a representative block
    classify each child by shape rules; split nested blocks; pair halves; test transposition
  end for
end for
label stems and heads; detect adapters, transducer joints, timestep MLPs by width matching
choose backbone, family, data-path order; derive d, L, f, E, V, KV bytes, active parameters
return anatomy with an evidence string for every decision

- **adaLN modulation:** a single matrix $[kd, t]$ from a width $t < d$, or $k \in \{6, 9, 12, 18\}$ when $t = d$, or a $[1, k, d]$ scale–shift table.

Three structural refinements matter in practice. If a child holds two non-leaf sub-modules that classify into different confident roles (an attention and an MLP inside one "inducing block"), it is split. Two half feed-forwards in sibling modules ($[f, d]$ in `intermediate`, $[d, f]$ in `output`) are paired. If a region is weak under the usual layout but becomes attention when every matrix is transposed, the weights are stored as $[\text{in}, \text{out}]$, as GPT-2's `Conv1D` does.

4.4 Stems, heads and the data path

Outside the blocks, a matrix $[V, d]$ with $V > d$ is a token table and its twin is an untied output head; a vector of length V is the head's bias; a small table next to a token table or convolutional stem is a position or type embedding. A matrix chain whose free input width equals one component's width d_X, or $k^2 d_X$ for $k \in \{2, 3, 4\}$ after pixel shuffling, and whose free output width equals another's d_Y, becomes an *adapter* with edges $X \rightarrow$ adapter $\rightarrow Y$.

Two other patterns get their own components. A *transducer* has a label embedding $[V, h_p]$ next to a recurrent cell, an encoder projection $[j, d_{\text{enc}}]$, and an output $[V', j]$ with $V \leq V' \leq V + 16$; the excess $V' - V$ is the number of duration bins of a token-and-duration transducer [27]. A *diffusion denoiser* has a timestep MLP from a sinusoidal width (128, 256 or 320) to a width t that feeds the modulation matrices in its blocks. Finally, a group of square maps $[C, C]$ with an aggregation matrix $[C, rC]$ is read as a message-passing layer with r reducers.

The backbone is the largest component that owns a token table or output head; the family (decoder, prefix LM, encoder–decoder, transducer, diffusion, encoder) follows from the components and edges. Algorithm 1 summarizes the procedure.

Derived quantities and cost. Parameters and stored bytes are sums. KV-cache bytes per token are $2 \sum_{\ell \in \text{attn}} x_{kv}^{(\ell)} \cdot b$; SSM layers add a constant state instead. With the top-k value from a config, active parameters per token are the non-expert parameters plus k/E of the experts. The whole procedure is a few passes over the tree, near-linear in the number of tensors. A median model takes 12.6 ms in a single JavaScript thread (95th percentile 63 ms, maximum 177 ms).

5 Benchmark

Corpus. `transformers` 5.22.1 [26] registers 788 configuration types. For each we instantiate the default configuration with the most specific model class available (conditional generation, sequence-to-sequence, transducer, causal LM, . . . , then the base model) on PyTorch's meta device, which creates shapes without memory. 548 types instantiate; after removing identical manifests, 511 remain. Tied parameters are counted once and persistent buffers are kept, as a saved checkpoint would. The corpus has 268,040 tensors (median 419 per model) and spans 1.5 K to 1.74 T parameters (median 473 M). 475 models contain attention, 73 mixtures of experts, 27 SSM or linear-attention mixers, 140 convolutional parts, 70 cross-attention; 231 have more than one stack and 153 have a stack with more than one block type.

Ground truth. Labels come from information the algorithm never sees: the Python class of every module. A tensor's role is given by its nearest ancestor class whose name ends in a recognizable suffix (`Attention`, `MLP`, `SparseMoeBlock`, `Mamba2Mixer`, `RMSNorm`, . . .), with any MoE container overriding its children and a short list of module-name fallbacks for loose tensors; tensors that match nothing are labeled *other* and excluded. Conv stems and position tables outside block stacks are merged into a single *input stem* role, because the library itself is inconsistent about which class owns them. Block lists are module lists whose children each hold at least two parameterized sub-modules, excluding expert containers and applying the same sub-layer convention as the algorithm; block types are the distinct class-composition signatures of the items. Hyperparameters come from the configuration (the text sub-configuration for multimodal models). Class names are themselves a noisy oracle, and we discuss the consequences in §8.

Metrics. Per-tensor role accuracy is weighted by parameter count within each model and averaged over models (macro, our headline, so that every architecture counts once) or pooled over all tensors (micro, dominated by the largest models). We also report per-role F1 over tensors, recall and precision of block lists by path, the fraction of matched lists with the exact block count and with the exact type partition, exact-match accuracy of hyperparameters, and exactness of the logical parameter total.

Conditions. *Names + shapes* is the full method. *Shapes only* replaces every non-numeric path token by a consistent random hash, which removes all lexical evidence while keeping the tree; it approximates checkpoints exported by unfamiliar code. *Quantized* rewrites every 2-D linear weight in NVFP4 (group 16), FP8 with per-channel scales, or GPTQ 4-bit with group 128. The *baseline* is a careful set of name-pattern rules of the kind most tools use, taken from the previous version of our own explorer: regular expressions for layer indices, `q_proj`/`query`/`c_attn`, `mlp`/`fc1`/`intermediate`, `encoder_attn`, `mixer`, and so on.

6 Results

Roles. Table 1 gives the main numbers. Per-tensor role accuracy is 96.4% macro. The per-model distribution is skewed: the median model is labeled perfectly, 411 of 511 models reach 99% and 468 reach 90%, while 10 fall below 50%. Name-pattern rules reach 69.2% macro, and their micro accuracy collapses to 8.4% because the parameter mass of the corpus sits in a handful of trillion-parameter MoE defaults whose experts they label as something else. With names hashed, the baseline has nothing to work with. Our method loses 1.7 points (371 models still reach 99%), so most of the work is done by shapes and structure rather than by vocabulary.

Figure 2 breaks the result down by role. Attention, feed-forward, MoE and norms are above 96 F1 with names and stay above 91 without them. Cross-attention drops from 98 to 71 without names. That is expected: when encoder and decoder share a width, the two attention regions of a decoder block have

	Role accuracy (%)		Block lists (%)				Params
Condition	macro	micro	recall	precision	count	types	exact
Names + shapes (ours)	**96.4**	**99.9**	98.6	91.4	100	92.5	511/511
Shapes only (ours)	94.7	99.8	98.6	91.4	100	92.5	511/511
Name patterns	69.2	8.4	–	–	–	–	–
Name patterns, names hashed	0.0	0.0	–	–	–	–	–
Ours, NVFP4 storage	96.4	99.9	98.6	91.4	100	92.5	511/511
Ours, FP8 storage	96.4	99.9	98.6	91.4	100	92.5	511/511
Ours, GPTQ storage	96.4	99.9	98.6	91.4	100	92.5	511/511
Ours, shapes only + NVFP4	94.7	99.8	98.6	91.4	100	92.5	511/511

Table 1: Main results on 511 architectures. *count*: matched lists with the exact number of blocks; *types*: matched lists whose block-type partition equals the ground truth. The name-pattern baseline produces no stacks. Micro accuracy is dominated by the three largest models, all trillion-parameter mixtures of experts: we label their experts correctly and the baseline does not.

	d	layers	FFN	experts	vocab	heads	KV heads	head size
All models	92.7	92.1	90.5	97.3	87.2	72.2	70.0	66.7
n	468	442	336	74	360	446	227	153
Single-stack models	95.0	96.3	94.1	96.4	85.6	70.3	63.9	57.1
Shapes only	92.7	92.1	90.5	94.6	84.7	72.2	70.0	66.7
Identifiable from shapes?	yes	yes	yes	yes	yes	only with per-head parameters		

Table 2: Exact-match accuracy (%) of hyperparameters against the configuration. FFN is evaluated on dense models only.

identical shapes, and only a name says which one reads the encoder. adaLN modulation (4 models) and recurrent cells (6 models) have too few examples for stable F1. Output heads score lowest, mostly because pooler and prediction-transform layers are labeled *head* by the class oracle but look like a feed-forward or embedding to the shape rules.

Blocks and types. 98.6% of ground-truth block lists (1232 of 1250) are found at the right path, and every matched list has the right number of blocks. Precision is lower (91.4%) because the algorithm also reports repeated sub-blocks inside convolutional backbones (ResNet bottleneck sub-layers, SEANet residual units) that the ground-truth convention does not count. The block-type partition is exact for 92.5% of matched lists; most misses are lists whose first block differs only in a module the class signature sees and the tensor signature does not, such as a relative-position bias owned by layer 0.

Hyperparameters. Table 2 separates identifiable from non-identifiable fields. Width, depth, FFN size and expert count are read correctly for 90–97% of models, and more often when a model has a single stack, which removes the ambiguity about which component the configuration describes. Head count and head size are right about 70% of the time. Using a prior (head size 128 for widths of 2048 and above, otherwise 64) is right more often (78%) than trusting a norm vector of plausible length (69%): in latent attention, the norm in front of the low-rank KV projection has the latent width, not the head width. We take this as a concrete instance of the identifiability limit in §3, and the tool labels both values as guesses unless a configuration confirms them.

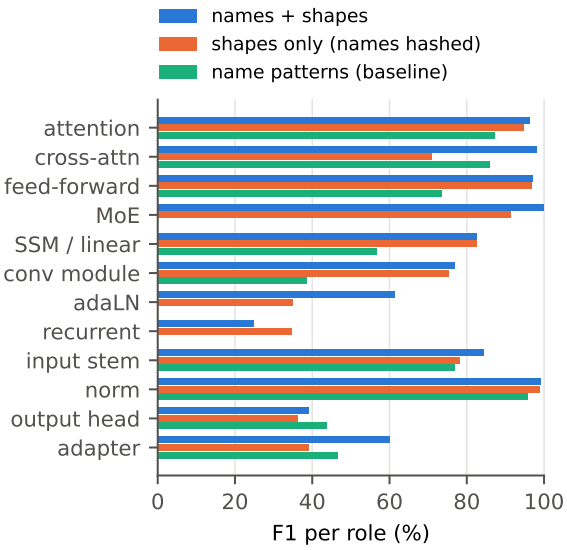


Figure 2: Per-role F1 over tensors.

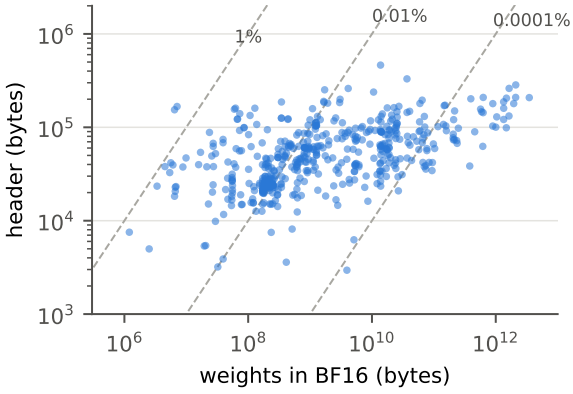


Figure 3: Header size against weight size for the 511 manifests (one 3 KB model lies off the left edge). Dashed lines mark constant ratios. The median header is 47 KB and the largest is 453 KB.

Quantized storage. Under NVFP4, FP8 and GPTQ storage the results are identical to full precision, and logical parameter totals stay exact for all 511 models (Table 1). This also holds with names hashed, because packed weights and their scales are recognized by dtype and shape ratios rather than by the suffix `weight_scale`. The point matters in practice: hub pages count packed bytes as parameters, so an NVFP4 model is often displayed at about 60% of its real size (§7).

Errors. The largest error groups are informative. ConvBERT's span-based dynamic convolution sits inside its attention module, so the region is labeled a convolution. The linear-attention layers of GLM-5-Next keep ordinary query, key and value projections and move the decay parameters into a sub-module, so the shape rules see attention; the opposite happens in EdgeTAM's perceiver, whose gated attention looks recurrent. ALBERT shares one layer across depth and stores no list at all, so nothing repeats and nothing is typed. I-BERT keeps integer copies of every weight, and the matching shapes imitate a gate stack; this also produces the only false transducer detection in the corpus, while the three real transducers are found. The ten models below 50% are ALBERT, I-BERT, xLSTM, XLNet, MobileBERT, MobileViTv2, PoolFormer, PatchTSMixer, NeoMME and an ASR encoder whose blocks are a single fused module; each departs from the block conventions the rules assume, rather than failing on a common architecture.

7 Case studies: nine models from 2026

These models were not in the benchmark, but they were not held out from development either: we studied them while extending the rules, and several components of §4 (transducer joints, timestep MLPs, message-passing layers, adapter components) were written because of them. Every change was then checked on the full benchmark for regressions. The case studies should therefore be read as demonstrations that the representation is rich enough for these architectures, not as an unbiased accuracy estimate; that role belongs to §6. Table 3 summarizes the results, and Figure 4 shows the tool on the first model.

Model	Family	Params	Header	Recovered anatomy (selected)
Nemotron 3.5 Super VL 120B-A12B	prefix LM	121.5 B	130 KB	ViT tower (32 L, d 1280) $\rightarrow$ projector $1280\times2^2{=}5120 \rightarrow 20480 \rightarrow 4096$ $\rightarrow$ 88-layer backbone: 40 SSM, 40 MoE (512 experts, 92.8% of weights), 8 attention (2 KV heads, 8 KB KV per token)
Nemotron 3.5 Transcribe 30B-A4B	prefix LM	32.4 B	154 KB	audio encoder (32 L, d 1280), diarization encoder (31 L, d 512), word-timestamp encoder, projector $1280 \rightarrow 2688$, 52-layer backbone: 23 SSM / 23 MoE (128 experts) / 6 attention; 4.36 B active with top-6
Nemotron 3.5 ASR streaming 0.6B	transducer	638 M	57 KB	24 conformer blocks (FFN, attention, conv, FFN); LSTM prediction net; joint $1024, 640 \rightarrow 640 \rightarrow 13088$, no duration bins (RNN-T)
Parakeet TDT 0.6B v3	transducer	627 M	63 KB	same encoder shape; joint output $8198 = 8193 + 5$ duration bins (TDT); parameter total within 24 of the hub's figure
Nemotron 3 Embed 1B NVFP4	encoder	1.14 B	41 KB	16 blocks, d 2048, 24/8 heads (guess), 112 NVFP4 linear layers (group 16), tied table; stored 1.03 GB
FastH3 Omni FP8 (transformer)†	diffusion	33.1 B	84 KB	2 refiner + 50 DiT blocks, 56 heads of 128 (from q/k norms), adaLN $18d$ (6 terms $\times$ 3 modalities), timestep MLP $256 \rightarrow 5376 \rightarrow 2688$; 300 FP8 tensors, 47.0 vs 66.3 GB
Kumo Tabular (large)†	encoder	214 M	41 KB	row embedding (6 induced + 6 row blocks, d 256) and 24-layer ICL backbone (d 1024); 854 MB in FP32 vs. 855 MB file
Kumo Relational†	encoder	29 M	24 KB	row embedding $\rightarrow$ message passing (4 square maps, 5 reducers) $\rightarrow$ 12-layer ICL backbone; 118 vs. 120 MB file

Table 3: Anatomies recovered from manifests of 2026 models (eight rows; the BF16 FastH3 variant gives the same structure). Manifests marked † are rebuilt from public source code and configuration because the reference implementation is not in `transformers`; the others are generated from the published `config.json` by the reference implementation, and NVFP4 storage follows the published quantization config.

A 121B hybrid. The Super VL model mixes three block types in one list. The type sequence reproduces the configuration's `layers_block_type` exactly. The projector's input width, 5120, is four times the vision width; the algorithm reports a 2×2 merge, which matches the configuration's `downsample_ratio` of 0.5. Experts hold 112.7 B of 121.5 B parameters; with the configured top-22 routing that gives 13.6 B active parameters per token, including the vision tower and both embedding tables, against the 12 B in the model's name. Because only 8 of 88 layers keep a KV cache, and those use 2 KV heads, the cache costs 8 KB per token in BF16. If all 88 layers used the same attention, the cache would be eleven times larger (88 KB per token).

Transducers. For both speech models the joint network is found from three shapes alone, and the count of duration bins falls out as a difference: Parakeet's joint output has 8198 rows and its label table 8193, giving the five durations $\{0, \ldots, 4\}$ in its configuration, while the streaming model's two sizes are equal (a standard RNN-T). Parakeet's parameter total differs from the hub's by 24, the number of batch-norm step counters, one per conformer block.

Packed weights. The NVFP4 embedding model has 1.14 B logical parameters, matching its model card's architecture section, while the hub's file statistics report 0.7 B because they count packed `U8` elements. That

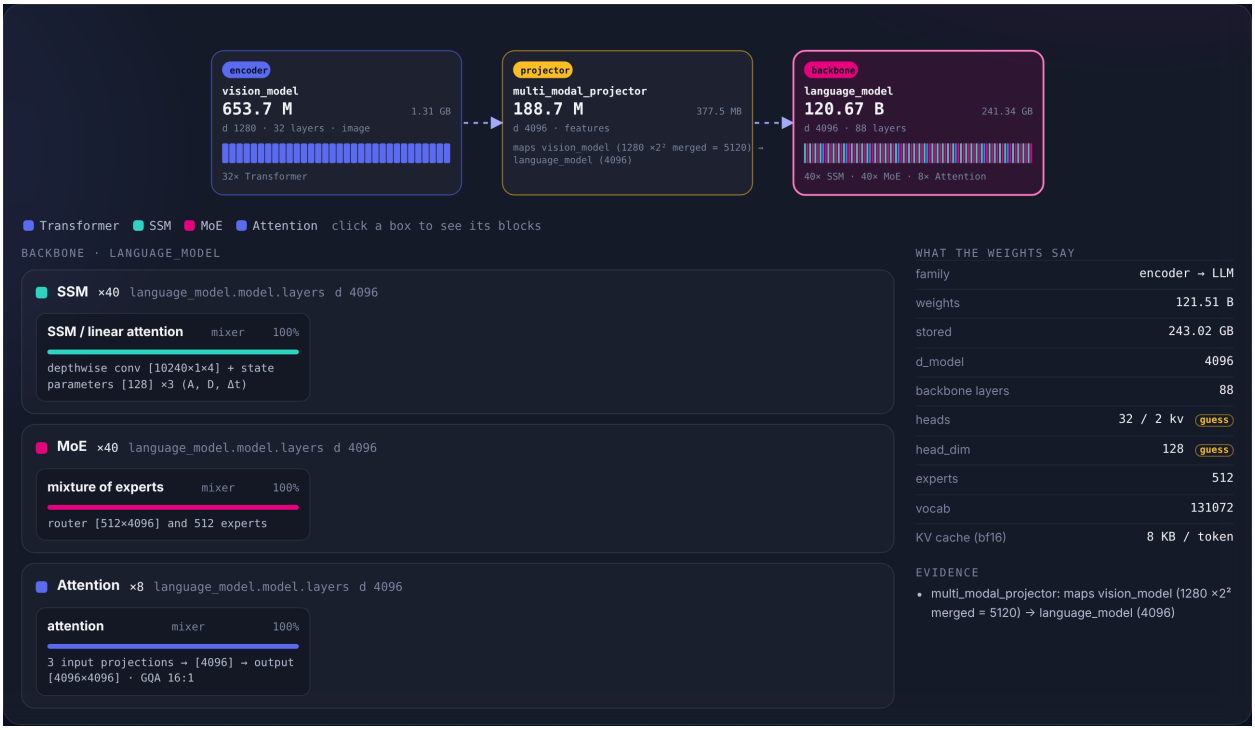


Figure 4: The open tool on the Super VL manifest. Components are drawn in data-path order with their block-type pattern; below, the regions of the selected component with the shape evidence behind each label, and on the right the values read from the weights. Head counts carry a *guess* badge (§3).

the model is an encoder (bidirectional, average pooling) cannot be seen in the weights: the tool reports "encoder, or a decoder with a tied head" unless `config.json` is available, where `is_causal: false` settles it.

What the case studies taught us. Besides the new components above, four fixes to existing rules came from these models: the residual width of a block is taken from its norms (the SSM inner width had won in mixed SSM/MoE blocks); `nn.Sequential` children are no longer split as sub-layers; a module that wraps two different sub-blocks is split; and in repositories with several top-level folders (diffusers layouts, transformer plus VAE) homes are not raised above the folder.

Limits seen here. The streaming ASR model's language-prompt projector maps a 1152-wide prompt table into the encoder, but the prompt table itself lives elsewhere, so the adapter is left unattached. The block-diffusion drafter that accompanies the Transcribe model is a separate repository and is not analysed.

8 Discussion and limitations

Ground truth. Class names are an independent signal, but not a perfect one. Some libraries place a position table inside a class called `...Embeddings`, others inside the encoder; some name a pooler `...Head`. We merged roles where the library is inconsistent (input stems) and kept a fallback list small, but the per-role numbers for heads and adapters should be read with this in mind. The headline numbers are robust to it because these roles carry little parameter mass.

One library's conventions. All corpus manifests come from `transformers` implementations, so they share naming habits. The shapes-only condition is our main protection against overfitting to those habits, and the 1.7-point drop suggests the structural rules transfer. Default configurations are not trained checkpoints, but the shapes, which are all the method reads, are the same.

Reconstructed manifests. Three case-study manifests were rebuilt from source because no reference implementation was available offline. Their totals match the published file sizes to within 2%, which supports the reconstructions, but module prefixes in the real checkpoints may differ. The tool itself reads the real headers when given a repository.

Scope. GGUF and ONNX files are not handled yet, though both carry comparable metadata. Old non-zip PyTorch pickles are rejected rather than parsed. Order inside a block and activation functions remain unknown, and FLOP estimates assume standard computation per role.

Responsible use. The method reads only what a hub already serves publicly and never downloads or executes weights. The PyTorch reader interprets a small, fixed subset of pickle opcodes as data and never resolves Python callables.

9 Conclusion

A tensor manifest is small, safe and almost always available, and it carries more of an architecture than one might expect. With a module tree, a few dozen shape rules and width matching, we recover blocks, block types, regions, data paths and logical sizes for 511 architectures at 96% role accuracy, keep most of that accuracy without any names, and are exact on parameter counts under modern quantized formats. The same analysis makes the limits explicit: head geometry, masking and routing are not in the shapes, and a tool should say so instead of guessing silently. We hope the benchmark and the open tool make it easier to look inside a model before committing to download it.

Availability. The tool, the inference code, the corpus builder and the evaluation harness are released under the MIT license at `https://github.com/JoesSattes/ai-model-explorer`; the tool runs at `https://joessattes.github.io/ai-model-explorer/`.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[2] Aeree Cho, Grace C. Kim, Alexander Karpekov, Alec Helbling, Zijie J. Wang, Seongmin Lee, Benjamin Hoover, and Duen Horng Chau. Transformer explainer: Interactive learning of text-generative models. *arXiv preprint arXiv:2408.04619*, 2024.

[3] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, 2024.

[4] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[5] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

[7] Google AI Edge. Model explorer: Intuitive and hierarchical visualization of model graphs. https://ai.google.dev/edge/model-explorer, 2024.

[8] Alex Graves. Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*, 2012.

[9] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

[10] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, and Ruoming Pang. Conformer: Convolution-augmented transformer for speech recognition. In *Interspeech*, 2020.

[11] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[12] Xing Hu, Ling Liang, Shuangchen Li, Lei Deng, Pengfei Zuo, Yu Ji, Xinfeng Xie, Yufei Ding, Chang Liu, Timothy Sherwood, and Yuan Xie. Deepsniffer: A dnn model extraction framework based on learning architectural hints. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 385–399, 2020.

[13] Hugging Face. safetensors: Simple, safe way to store and distribute tensors. https://github.com/huggingface/safetensors, 2022.

[14] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems*, 2024.

[15] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022.

[16] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, pages 220–229, 2019.

[17] NVIDIA. Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models. *arXiv preprint arXiv:2504.03624*, 2025.

[18] NVIDIA. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149*, 2025.

[19] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

[20] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. TabICL: A tabular foundation model for in-context learning on large data. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 50817–50847, 2025.

[21] Dima Rekesh, Nithin Rao Koluguri, Samuel Kriman, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg. Fast conformer with linearly scalable attention for efficient speech recognition. In *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023.

[22] Lutz Roeder. Netron: Visualizer for neural network, deep learning and machine learning models. https://github.com/lutzroeder/netron, 2017.

[23] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[24] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

[25] Sattaya Singkul. AI Model Explorer. https://joessattes.github.io/ai-model-explorer/, 2026.

[26] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020.

[27] Hainan Xu, Fei Jia, Somshubra Majumdar, He Huang, Shinji Watanabe, and Boris Ginsburg. Efficient sequence transduction by jointly predicting tokens and durations. In *International Conference on Machine Learning*, 2023.

[28] Mengjia Yan, Christopher W. Fletcher, and Josep Torrellas. Cache telepathy: Leveraging shared resource attacks to learn DNN architectures. In *29th USENIX Security Symposium*, pages 2003–2020, 2020.

[29] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

[30] Boyi Zeng, Lizheng Wang, Yuncong Hu, Yi Xu, Chenghu Zhou, Xinbing Wang, Yu Yu, and Zhouhan Lin. HuRef: HUman-REadable fingerprint for large language models. In *Advances in Neural Information Processing Systems*, 2024.

[31] Jian Zhao, Shenao Wang, Yanjie Zhao, Xinyi Hou, Kailong Wang, Peiming Gao, Yuanchao Zhang, Chen Wei, and Haoyu Wang. Models are codes: Towards measuring malicious code poisoning attacks on pre-trained model hubs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

A Corpus construction details

The model class for each configuration type is chosen in this order: image-text-to-text, multimodal LM, speech sequence-to-sequence, sequence-to-sequence LM, TDT, RNN-T, causal LM, CTC, masked LM, image classification, text-to-waveform, object detection, base model. Instantiation uses `_from_config` under `torch.device("meta")` with a 90-second limit. Of the 240 types that do not instantiate, 177 are sub-configurations with no model class of their own (for example vision or audio towers registered separately), and the rest fail on required sub-configs or optional dependencies. Manifests are deduplicated by the exact list of names and shapes.

B Synthetic quantized storage

For every 2-D tensor named `*.weight` whose input dimension is a multiple of 32 and output a multiple of 8, excluding embeddings, heads and norms: NVFP4 replaces $[o, i]$ by `U8` $[o, i/2]$, `F8_E4M3` $[o, i/16]$ and two `F32` scalars; FP8 replaces it by `F8_E4M3` $[o, i]$ and `F32` $[o, 1]$; GPTQ by `qweight I32` $[i/8, o]$, `qzeros` $[i/128, o/8]$, `scales F16` $[i/128, o]$ and `g_idx` $[i]$. Under name hashing these suffixes are hashed too, so recognition relies on dtypes and shape ratios.